decide which one, among the possible solutions, is the actual best choice.

- Or, we can replace the predicate passed to our nearest neighbor method: for instance, as shown in listing 11.7, instead of a unary predicate using a binary function that takes as arguments the current node and the best solution found so far, and returns which one is the best.

And, of course, we can also use a combination of the two methods.

The first solution doesn't ask for any change to our container, so we won't need to develop it any further: you can use sorting, a priority queue, or any selector algorithm you like, in order to decide which solution is the best, according to your business rules. This mechanism has an advantage: it allows you to also try solutions where you order different items at different shops, and see if it works better than the best "all-at-one-place" solution.

Listing 11.7 Another version of the filteredNearestNeighbor method

```
function fNN(node, location, cmpShops, (nnDist, nn)=(inf,
null))     #1
  if node == null
then                                                    #2
    return (nnDist, nn)

else
     #3
    dist ← distance(node.shop.point,
location)                                  #4
```